F1 AI — Team Assignments

Project goal

Build a 2D F1 driving simulator, train an AI using one driver's data across multiple circuits, and see how fast it can complete a lap on a track it has never seen before.

Shared conventions

- All coordinates and distances: **metres**; speed: **m/s**; time: **seconds**; angles: **radians**.
- World coordinates use **x right, y up**, and positive angles rotate counterclockwise.
- Positive lateral offset and curvature mean **left** relative to the direction of travel.
- `steering` is the **front-wheel steering angle in radians**; `throttle` and `brake` are each in **[0, 1]**.
- The schemas below are **handoff contracts**. Numbers are **illustrative**, not actual Monza measurements or simulated predictions.
- Teammates are free to choose their implementations while preserving these interfaces.

Shared API — Tony's track module

```
from track import Track

track = Track("monza")

track.length               # 5793.0  (illustrative)
track.position(500.0)      # (120.0, 80.0)
track.heading(500.0)       # 1.57
track.curvature(500.0)     # 0.02
track.boundaries(500.0)    # ((114.0, 80.0), (126.0, 80.0))
track.left_width(500.0)    # 6.0
track.right_width(500.0)   # 6.0
track.project(118.0, 80.0) # (500.0, 2.0)
```

`position(s)` samples centerline coordinates, while `project(x, y)` finds the closest track distance and signed lateral offset. `s` wraps around a closed circuit. This API describes **track geometry, not vehicle motion**.

Ownership and handoffs

Person	Owner / component	Receives	Provides
1	**Track geometry**	Circuit centerline and widths	`track.py` and `Track` object
2	**Vehicle physics**	Car state, action, timestep	Updated car state
3	**Simulator**	`Track`, physics update, action	New state, race info, AI observation, rendering
4	**F1 telemetry**	FastF1 session data	Clean lap CSV / records
5	**Data alignment**	Lap records and `Track`	Track-relative driver records
6	**Imitation learning**	Aligned lap records and `Track`	Throttle/brake predictor
7	**Baseline & evaluation**	Observations, simulator, controller	Driving actions and comparable lap metrics

---

Tony — Track geometry

Libraries: `numpy`, `scipy`, `matplotlib`.

Responsibility: Build and own the `Track` API above. **Assume completed for everyone else's first assignment.**

Input — circuit centerline and widths:

```
track_points = [
    # x, y, left_width, right_width (metres)
    (100.0, 50.0, 6.0, 6.0),
    (110.0, 50.0, 6.0, 6.0),
    (120.0, 55.0, 7.0, 6.0),
]
```

Expected output — the shared `Track` API at the top of this note.

○ Load the agreed circuit.
○ Plot continuous centerline, left boundary and right boundary.
○ Validate continuity, units, and `project(x, y)`.

Done when

Other components can query position, heading, curvature, widths, boundaries and projection at arbitrary track distances.

---

2. Vehicle physics — Car movement

Libraries: `numpy`, `pytest`.

Responsibility: Simulate car movement in an empty 2D plane. A kinematic bicycle model is a suggested starting point; implementation details are up to this owner. **No track dependency.**

Function to implement

```
def update_car(state: dict, action: dict, dt: float) -> dict:
    """Return car state after dt simulated seconds."""
```

Example input

```
state = {
    "x": 120.0,       # metres
    "y": 80.0,        # metres
    "heading": 0.0,   # radians
    "speed": 20.0     # m/s
}

action = {
    "steering": 0.0,  # front-wheel angle, radians
    "throttle": 1.0,  # 0..1
    "brake": 0.0      # 0..1
}

dt = 0.1
new_state = update_car(state, action, dt)
```

Expected output

```
new_state = {
    "x": 122.02,
    "y": 80.0,
    "heading": 0.0,
    "speed": 20.4
}
```

Illustrative only: actual values depend on the physics parameters and numerical integration scheme.

First task

○ Implement acceleration, braking and steering.
○ Test straight-line acceleration and braking.
○ Test approximately circular motion with constant steering at a controlled speed.

Done when

Identical input states, actions and timesteps produce reproducible updated states; no circuit is needed.

---

3. Simulator — Environment & visualization

Libraries: `pygame`, `numpy`. **Do not use Gymnasium.**

Responsibility: Run a driving session by combining `Track` geometry and the physics update; display the car; track lap progress and elapsed time; detect road departures.

Available inputs / API calls

```
# From Tony
track.position(s)
track.heading(s)
track.curvature(s)
track.boundaries(s)
track.left_width(s)
track.right_width(s)
track.project(x, y)
track.length

# From vehicle physics
new_state = update_car(state, action, dt)
```

Functions to implement

```
def reset_simulation(track) -> tuple[dict, dict]:
    """Return (starting_car_state, fresh_race_info)."""


def simulate_step(state: dict, action: dict, dt: float, track) -> tuple[dict, dict]:
    """Advance motion; update race status; return (new_state, race_info).

    Keep previous lap progress and time in simulator-owned session data.
    """


def get_observation(state: dict, track) -> dict:
    """Build the agreed input features for driving controllers."""


def render(track, state: dict) -> None:
    """Draw the track and car; show timing and progress."""
```

Session state

`race_info` cannot be calculated from only the current car state. The simulator must retain elapsed time, previous track position, cumulative forward progress, and lap count between updates. These may be held in a simulator object or session context internally, without changing the agreed external data formats.

Example input

```
state = {"x": 118.0, "y": 80.0, "heading": 1.57, "speed": 20.0}
action = {"steering": 0.05, "throttle": 0.8, "brake": 0.0}
dt = 0.1

new_state, race_info = simulate_step(state, action, dt, track)
observation = get_observation(new_state, track)
render(track, new_state)
```

Expected output

```
new_state = {
    "x": 117.99,
    "y": 82.02,
    "heading": 1.61,
    "speed": 20.4
}

race_info = {
    "distance": 502.0,       # progress along centerline (metres)
    "elapsed_time": 25.5,    # seconds since reset
    "off_track": False,
    "lap_complete": False
}

observation = {
    "speed": 20.4,
    "lateral_offset": 2.0,
    "heading_error": 0.04,   # car heading minus track heading
    "curvature_ahead": [0.02, 0.025, 0.03]
}
```

Feature convention: `curvature_ahead` samples at offsets **[0, 10, 20] metres** from the projected car location, in that order. Heading error should be wrapped to the interval **[-π, π)**. The example outputs are schematic and not guaranteed to correspond to a particular physical trajectory.

First task

○ Render a simple circular track and movable car; mock physics until Person 2's module arrives.
○ Implement reset and repeated stepping, elapsed time and lap progress.
○ Detect track-limit violations using lateral offset and left/right widths.
○ Return the exact `observation` and `race_info` schemas above.

Done when

A driver or scripted action sequence can move around the circuit, and each step yields car state, lap information and the agreed observation fields.

---

4. F1 telemetry — Data extraction

Libraries: `fastf1`, `pandas`, `matplotlib`.

Responsibility: Retrieve a complete real F1 qualifying lap and prepare clean, synchronized driver telemetry.

Function to implement

```
def extract_lap(year: int, circuit: str, session_type: str, driver: str) -> list[dict]:
    """Return ordered samples with time, x, y, speed, throttle, brake."""
```

Example input / library call

```
import fastf1

session = fastf1.get_session(2025, "Monza", "Q")
session.load()

lap = session.laps.pick_drivers("VER").pick_fastest()
telemetry = lap.get_telemetry()

lap_data = extract_lap(2025, "Monza", "Q", "VER")
```

Expected output

```
lap_data = [
    {
        "time": 0.0, "x": 1200.0, "y": 800.0,
        "speed": 20.0, "throttle": 0.8, "brake": 0.0
    },
    {
        "time": 0.1, "x": 1200.0, "y": 802.0,
        "speed": 20.4, "throttle": 0.8, "brake": 0.0
    }
]
```

Normalization: Speed in **m/s**, throttle as **0–1**, brake as **0.0 or 1.0** if using FastF1's boolean brake indicator. `time` is seconds since start of lap. Position uses the telemetry's own coordinate frame, **not** Tony's track coordinates. Do not claim brake pressure or steering angle is measured.

First task

○ Export one qualifying lap into a CSV with these field names and units.
○ Verify timestamps increase and plot speed against lap distance.
○ Document missing data and any interpolation/resampling.

Done when

Another person can load the CSV without knowing anything about FastF1's native column names or units.

---

5. Data alignment — Match driver coordinates to track

Libraries: `numpy`, `scipy`, `pandas`, `matplotlib`.

Responsibility: Convert a telemetry trajectory into the simulator's world coordinate system; express it relative to Tony's track.

Available inputs / API calls

```
lap_data = extract_lap(2025, "Monza", "Q", "VER")

track.position(s)
track.heading(s)
track.project(x, y)
track.length
```

Functions to implement

```
def align_coordinates(lap_data: list[dict], track) -> list[dict]:
    """Transform an entire driver's X/Y trajectory into track world coordinates."""


def convert_to_track_coordinates(aligned_data: list[dict], track) -> list[dict]:
    """Project positions to s / lateral offset; estimate driver heading error."""
```

Example input

```
lap_data = [
    {
        "time": 0.1, "x": 1200.0, "y": 800.0,
        "speed": 20.4, "throttle": 0.8, "brake": 0.0
    }
]

# Actual alignment needs a full trajectory, not one sample.
aligned_data = align_coordinates(lap_data, track)
training_data = convert_to_track_coordinates(aligned_data, track)
```

Expected output

```
aligned_data = [
    {
        "time": 0.1, "x": 118.0, "y": 80.0,
        "speed": 20.4, "throttle": 0.8, "brake": 0.0
    }
]

training_data = [
    {
        "time": 0.1,
        "s": 500.0,
        "lateral_offset": 2.0,
        "heading_error": 0.0,
        "speed": 20.4,
        "throttle": 0.8,
        "brake": 0.0
    }
]
```

`heading_error` requires a heading estimate from **several consecutive driver positions**, then comparison with `track.heading(s)`. The coordinate transform must be estimated from a substantial trajectory and checked visually; a single point cannot determine it.

First task

○ Generate a synthetic trajectory and apply a known translation/rotation/scale.
○ Recover the transformation and overlay transformed points on a reference track.
○ Convert transformed data to the standardized track-relative records.

Done when

Full real driver trajectories align with the track, progress follows the lap, and lateral offsets are plausible.

---

6. AI — Imitation learning

Libraries: `scikit-learn`, `pytorch`, `numpy`, `pandas`.

Responsibility: Predict the driver's **throttle and brake** from track-relative observations. Start with supervised baselines; steering labels are not available from FastF1.

Available inputs / API calls

```
# From alignment
aligned_laps = [
    {
        "time": 0.1, "s": 500.0,
        "lateral_offset": 2.0, "heading_error": 0.0,
        "speed": 20.4, "throttle": 0.8, "brake": 0.0
    }
]

# From Tony
track.curvature(s)
```

Functions to implement

```
def prepare_training_data(aligned_laps: list[dict], track):
    """Return (X, y), where X is observations and y is throttle/brake labels."""


def train_model(X, y):
    """Train and return a model."""


def predict_action(model, observation: dict) -> dict:
    """Return {'throttle': float, 'brake': float}."""
```

Example input

```
aligned_laps = [
    {
        "time": 0.1, "s": 500.0,
        "lateral_offset": 2.0, "heading_error": 0.0,
        "speed": 20.4, "throttle": 0.8, "brake": 0.0
    }
]

X, y = prepare_training_data(aligned_laps, track)
model = train_model(X, y)
```

Expected output

```
X = [
    {
        "speed": 20.4,
        "lateral_offset": 2.0,
        "heading_error": 0.0,
        "curvature_ahead": [0.02, 0.025, 0.03]  # s+[0,10,20] m
    }
]

y = [{"throttle": 0.8, "brake": 0.0}]

predicted_action = predict_action(model, X[0])
# {"throttle": 0.75, "brake": 0.0}  (illustrative)
```

Steering is out of scope for this first model

The output is **not yet a complete simulator action**. The baseline/controller component provides steering while we investigate labels or trajectory-based approaches. F1 telemetry alone does not provide measured steering angles.

First task

○ Implement data preparation on synthetic samples matching the schema.
○ Train a simple `scikit-learn` model before trying a neural network.
○ Report predictions and held-out performance; split by lap/session where possible to avoid adjacent-sample leakage.

Done when

The model can be loaded and queried with a simulator observation and returns documented throttle/brake predictions.

---

7. Baseline controller & evaluation

Libraries: `numpy`, `pandas`, `matplotlib`.

Responsibility: Build a simple rule-based driver and measure whether learned controllers improve results. **RL comes later.**

Available inputs / API calls

```
# From simulator
state, race_info = reset_simulation(track)
new_state, race_info = simulate_step(state, action, dt, track)
observation = get_observation(new_state, track)

# From Tony
track.heading(s)
track.curvature(s)
track.project(x, y)

# From imitation learning (throttle/brake only)
learned_controls = predict_action(model, observation)
```

Functions to implement

```
def baseline_action(observation: dict) -> dict:
    """Return steering, throttle, brake controls from simple driving rules."""


def evaluate_controller(controller, track, dt: float = 0.1) -> dict:
    """Run a timed attempt and return comparable lap metrics."""
```

Example input

```
observation = {
    "speed": 20.0,
    "lateral_offset": 2.0,
    "heading_error": 0.0,
    "curvature_ahead": [0.02, 0.025, 0.03]
}

action = baseline_action(observation)
results = evaluate_controller(baseline_action, track, dt=0.1)
```

Expected output

```
action = {
    "steering": 0.05,
    "throttle": 0.6,
    "brake": 0.0
}

results = {
    "controller": "baseline",
    "completed_lap": True,
    "lap_time": 105.3,
    "off_track_events": 0,
    "distance_travelled": 5793.0
}
```

Numbers are placeholders, not measured performance. The evaluator must handle timeouts and failed laps. When comparing a throttle/brake model, use the **same steering controller** for both the learned model and its baseline.

First task

○ Produce steering, throttle and brake from a dummy observation.
○ Design a conservative centerline-following controller.
○ Define comparable evaluation metrics and failure rules.
○ Once the simulator is connected, attempt one valid lap.

Done when

A deterministic controller can be benchmarked against another under identical simulation conditions, reporting completion, lap time and track-limit violations.

Deliverables and file names

Person	Deliverable
1 — Tony	`track.py`
2 — Physics	`physics.py` + tests
3 — Simulator	`simulator.py` + rendering demo
4 — Telemetry	`telemetry.py` + clean lap CSV
5 — Alignment	`alignment.py` + overlay plot
6 — Imitation	`imitation.py` + saved model + validation results
7 — Baseline/evaluation	`baseline.py`, `evaluation.py` + benchmark output